

Functions in JavaScript

Definition

A function is a **reusable block of code** designed to perform a specific task.

Functions can be defined once and **invoked (called)** whenever needed throughout your application.

```
function sayHello() {  
  console.log("Hello!");  
}
```

Key Use Cases

- **Code Reusability:** Write once, execute many times without duplication.
- **Organization:** Breaks complex programs into smaller, manageable pieces.
- **Abstraction:** Hide implementation details and focus on the result.
- **Callbacks:** Pass functions as arguments to handle asynchronous tasks.

Declaring vs. Invoking Functions

1. Function Definition

The blueprint where you specify the name, parameters, and code block.

```
function functionName() {  
  // logic here  
}  
  
function add(a, b) {  
  return a + b;  
}
```

2. Function Call

Execution of the stored block by referencing the name and providing arguments.

```
// No arguments  
functionName();  
  
// With arguments  
add(5, 10);
```

Also known as "Invoking" or "Executing" a function.

Arrow Functions in Javascript

Syntax Overview

A concise way to write functions in ES6+, replacing the **function** keyword with **=>**.

```
// Standard Arrow Function  
const greet = (name) => {  
  return `Hello, ${name}`;  
};
```

Key Advantages

- **Implicit Returns:** One-liners don't need curly braces or 'return'.
- **Lexical 'this':** They inherit 'this' from the surrounding scope.
- **Modern Code:** Perfect for array methods like map and filter.
- **Brevity:** Reduces boilerplate code significantly.

The forEach Loop in Arrays

Syntax & Purpose

The **forEach** method executes a provided function once for each array element.

```
arr.forEach(callbackfunction)
```

The Callback Function:

A function passed as an argument to another function, executed for every item in the array.

Implementation

Commonly used with arrow functions for cleaner, more readable code.

```
// Example usage
arr.forEach((val) => {
  console.log(val);
});
```

Key Characteristic:

Unlike traditional for loops, forEach provides a more functional approach to iteration.

Higher-Order Functions (HOF)

What is a HOF?

Functions that operate on other functions, either by taking them as **arguments** or by **returning** them.

Core Concepts:

- Abstraction of actions, not just values.
- Enables functional programming patterns.
- Promotes "Don't Repeat Yourself" (DRY).

Examples in JS

Array Methods:

```
.map(), .filter(), .reduce()
```

Code Sample:

```
const twice = (f, x) => f(f(x));  
const add3 = x => x + 3;  
twice(add3, 7); // Result: 13
```

Note: Functions are "First-Class Citizens" in JS.

The .map() Method in JavaScript

Definition & Concept

Creates a **new array** by calling a function for every array element. It does not change the original array.

```
const newArr = arr.map(callback);
```

Key Features:

- Non-mutating method.
- Always returns a new array of the same length.

Practical Example

Doubling numbers in an array using an arrow function.

```
const nums = [1, 2, 3];  
const doubled = nums.map(n => n * 2);  
// doubled is [2, 4, 6]  
// nums remains [1, 2, 3]
```

Transformation is the primary use case for .map().

The .filter() Method in JavaScript

Definition & Logic

Filters an array by creating a **new array** with elements that pass a specific test (truthy condition).

```
const result = arr.filter(callback);
```

Key Rules:

- Returns a shallow copy.
- Skips empty array elements.
- Original array is untouched.

Practical Example

Extracting even numbers from a list using a predicate function.

```
const nums = [1, 2, 3, 4, 5];  
  
const evens = nums.filter(n => n % 2 === 0);  
  
// evens is [2, 4]  
  
// nums is still [1, 2, 3, 4, 5]
```

Conditional selection is the primary use case for .filter().

The .reduce() Method in JavaScript

Definition & Purpose

Performs operations on an array to **reduce** it to a **single value**.

```
arr.reduce(callback, initialValue)
```

Key Components:

- Accumulator: Stores the result.
- Current Value: Current element being processed.
- Initial Value: Starting value of the accumulator.

Practical Example

Calculating the sum of all numbers in an array.

```
const nums = [1, 2, 3, 4];  
const sum = nums.reduce((res, curr) => {  
  return res + curr;  
}, 0);
```

Summation and finding max/min are common use cases for .reduce().

Q1: Student Marks Analysis

Input Data

Given an array of student objects:

```
const students = [  
  { name: "Omkar", marks: 85 },  
  { name: "Rohit", marks: 92 },  
  { name: "Amit", marks: 78 },  
  { name: "Sneha", marks: 96 },  
  { name: "Priya", marks: 88 }  
];
```

Required Tasks

Using **filter()**, **map()**, and **reduce()**:

- 1 Filter students who scored **80 or more**.
- 2 Create a new array containing **only their names**.
- 3 Find the **average marks** of the filtered students.
Chain these methods for a clean solution.

Q2: Number Transformation Challenge

Input Data

Given the following array:

```
const nums = [3, 8, 12, 5, 20, 7, 15];
```

This array contains various integers to be transformed through a chain of methods.

Challenge Steps

Chain **filter()**, **map()**, and **reduce()**:

- 1 Keep numbers divisible by **3 or 5**.
- 2 Square each selected number.
- 3 Find the sum of all squared values.

Expected Output: A single number.